For these maps to exist, and be learned, $\mathbf{h}_t$ must jointly optimize toward a belief state—a sufficient statistic for the history to predict the future.

Remark. Optimizing only next-token consistency (i.e., Equation 1) in standard autoregressive transformers does not guarantee that $\mathbf{h}_t$ forms a belief state (see Theorem 3 in Hu et al. [2025]). Intuitively, self-attention enables ad-hoc lookup of past tokens, so there is no pressure to compress all necessary information about the past into compact latent summaries at every time step.

Better Data Efficiency. A distinctive feature of NextLat is the richness of its learning signal. In prior methods that supervise in token space (see Figure 1), the learning signal is only anchored to the next token, or multiple tokens. NextLat additionally supervises in latent space. Specifically, the model is trained to predict its own next hidden state $\mathbf{h}_{t+1}$, which parameterizes the full predictive distribution over X_{t+2}. This shifts supervision from individual one-hot token labels to distribution-level alignment. Moreover, because the latent dynamics compose recursively—each latent is trained to predict the next—$\mathbf{h}_{t+1}$ implicitly carries information about future states $\mathbf{h}_{t+2}, \mathbf{h}_{t+3}, \ldots$. As a result, NextLat not only provides learning signals that are dense in the vocabulary space, but also propagates information about future tokens into earlier representations. By augmenting sparse one-hot targets with dense latent-state supervision, NextLat extracts more learning signal from each training sequence, leading to improved data efficiency.

3.2 Learning to Predict Next-Latent States

We now describe the practical implementation of NextLat, which augments standard next-token prediction with a self-supervised predictions in the latent space. Our NextLat implementation operates primarily on the hidden states (i.e., the final-layer outputs) as they provide compact, fixed-dimensional vectors through which gradients can be propagated through the entire transformer efficiently. As usual, we optimize the transformer and output head for next-token prediction (Equation 1) using the cross-entropy loss:

$$\mathcal{L}_{\text{next-token}}(\theta) = \mathbb{E}_{t<T}\big[-\log p_\theta(X_{t+1} \mid \mathbf{h}_t)\big].$$

NextLat additionally enforces transition consistency (Equation 2) of the hidden states by introducing a latent dynamics model p_ψ that predicts the next hidden state $\mathbf{h}_{t+1}$ directly from $(\mathbf{h}_t, X_{t+1})$. For a deterministic transformer model, $\mathbb{P}(\mathbf{h}_{t+1} \mid \mathbf{h}_t, X_{t+1})$ is a Dirac distribution, and we can optimize p_ψ via regression[3]. Moreover, observe that an ideal latent dynamics model should admit recursive consistency: its one-step map should compose correctly across multiple steps. Let $\hat{\mathbf{h}}_{t+d} = p_\psi(\mathbf{h}_t, X_{t+1:t+d})$ denote the recursive rollout of p_ψ over a d-step horizon using teacher-forced tokens $X_{t+1:t+d}$. We supervise all d intermediate rollouts using the Smooth L1 loss:

$$\mathcal{L}_{\text{next-h}}(\theta, \psi; d) = \mathbb{E}_t\Big[\frac{1}{d}\sum_{i=1}^{d} \text{SmoothL1Loss}\big(\mathbf{sg}[\mathbf{h}_{t+i}], \hat{\mathbf{h}}_{t+i}\big)\Big], \tag{3}$$

where $\mathbf{sg}[\cdot]$ denotes the stop-gradient operator, used to prevent representational collapse in self-predictive learning [Ni et al., 2024][4]. **Note that belief state convergence (i.e., Theorem 3.2) already holds for $d = 1$. Multi-step supervision serves only to provide richer learning signal.** To further align the semantics of predicted states $\hat{\mathbf{h}}$ with true states, we introduce a complementary KL objective enforcing agreement in token-prediction space:

$$\mathcal{L}_{\text{KL}}(\theta, \psi; d) = \mathbb{E}_t\Big[\frac{1}{d}\sum_{i=1}^{d} D_{\text{KL}}\Big(p_\theta^{\mathbf{sg}}(\cdot \mid \mathbf{sg}[\mathbf{h}_{t+i}]) \,\|\, p_\theta^{\mathbf{sg}}(\cdot \mid \hat{\mathbf{h}}_{t+i})\Big)\Big], \tag{4}$$

where the output head $p_\theta^{\mathbf{sg}}(\cdot)$ is frozen so that gradients flow only through the latent dynamics model. This KL acts similarly to *knowledge distillation* [Hinton et al., 2015], providing soft supervision that guides learning of p_ψ. It also resembles *observation reconstruction* in self-predictive RL [Subramanian et al., 2022, Ni et al., 2024], encouraging $\hat{\mathbf{h}}_{t+i}$ to reproduce the distribution over next observations (i.e., the output head's logits).

[3] If considering a stochastic transformer model, p_ψ can be optimized through variational inference.

[4] Technically speaking, the next-token prediction objective already provides grounding against representational collapse. However, in our ablations, we empirically observe better performance when applying the stop-gradient.

Overall Objective. The final NextLat objective combines all components, minimizing the following loss:

$$\mathcal{L}_{\text{NextLat}}(\theta, \psi; d, \lambda_{\text{next-h}}, \lambda_{\text{KL}}) = \mathcal{L}_{\text{next-token}}(\theta) + \lambda_{\text{next-h}}\, \mathcal{L}_{\text{next-h}}(\theta, \psi; d) + \lambda_{\text{KL}}\, \mathcal{L}_{\text{KL}}(\theta, \psi; d), \tag{5}$$

where $\lambda_{\text{next-h}}, \lambda_{\text{KL}} > 0$ are scalar coefficients. Importantly, during inference, the learned transformer can decode independently; p_ψ is only needed during training to shape the transformer representations.

In the experiments that follow, we parameterize p_ψ using simple MLPs as our goal is to demonstrate that NextLat yields significant performance gains over baselines even without sophisticated latent dynamics architectures. Additional implementation details and ablations of key NextLat design choices are provided in Sections C and D, respectively. To illustrate the simplicity of our approach, we also include a PyTorch-style pseudocode of the NextLat objective in Algorithm 1.

3.3 Variable-Length Self-Speculative Decoding

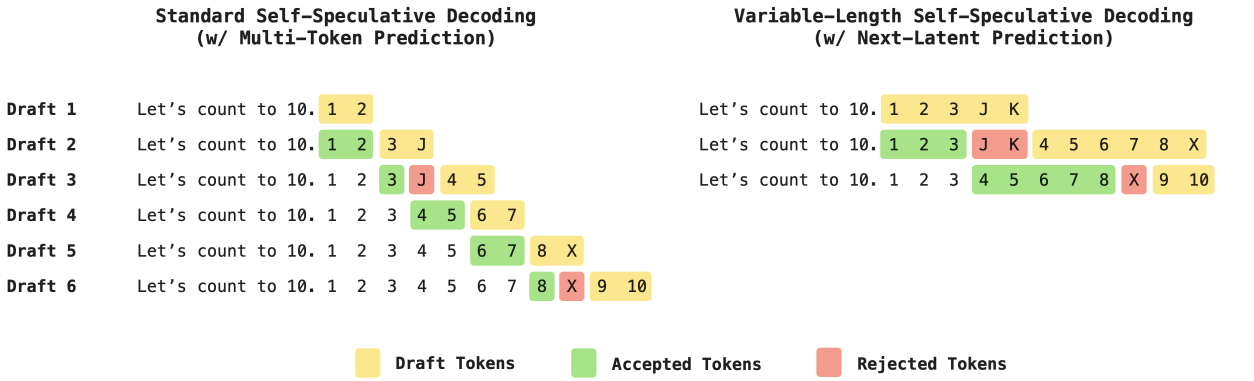


Figure 2: Illustration comparing self-speculative decoding with multi-token prediction (MTP) vs. NextLat. MTP uses a fixed draft length per step (e.g., $d = 2$ tokens here), whereas NextLat enables *variable-length self-speculative decoding*, reducing draft-verification cycles and accelerating inference.

In Figure 2, we illustrate the difference in self-speculative decoding capabilities between MTP methods and NextLat[5]. MTP models are trained to predict the next d tokens and are therefore limited to drafting at most d tokens per draft–verification cycle. In contrast, even when trained with just $d = 1$ (i.e., no multi-step supervision), NextLat can recursively compose predictions via its latent dynamics:

$$\mathbf{h}_t \xrightarrow[\text{decode token}]{p_\theta} X_{t+1} \xrightarrow[\text{update state}]{p_\psi} \mathbf{h}_{t+1} \xrightarrow[\text{decode token}]{p_\theta} X_{t+2} \xrightarrow[\text{update state}]{p_\psi} \mathbf{h}_{t+2} \cdots,$$

enabling variable-length drafting during self-speculative decoding. We demonstrate in Section 4.4 that, even with shallow training horizons ($d = 1, 2$) in the language domain, NextLat's latent dynamics remain coherent far beyond the training horizon, enabling longer drafts and much faster inference than MTP baselines.

4 Experiments

Modeling coherent latent dynamics and compact beliefs about the underlying data-generating process is fundamental to both algorithmic and human reasoning. Therefore, in this section, we evaluate NextLat on four key axes where such capabilities matter most: world modeling, reasoning, planning, and language modeling.

[5]The figure illustrates the potential for adaptive draft lengths with NextLat. In the scope of this work, however, we do not implement adaptive drafting strategies; once selected, the draft length remains static throughout decoding. Instead, we exploit the ability to *vary* these fixed draft lengths to extend beyond the training horizon d in order to maximize inference speedup.

Our baseline comparisons include transformer-based belief-learning methods, i.e., BST [Hu et al., 2025] and JTP [Ahn et al., 2025]. Further discussions and detailed comparisons with these methods are provided in Section 5 and Appendix A. For completeness, we also report the performances of standard next-token prediction (GPT) and multi-token prediction (MTP). The MTP baseline follows the implementation of Gloeckle et al. [2024], and we follow the `nanoGPT` codebase [Karpathy, 2022] for our decoder-only transformer implementations. Hereafter, we use the term "horizon" to refer to the multi-step prediction horizon d in JTP, MTP and NextLat, and we match horizon across these methods in all experiments to ensure fair comparisons. For specific experiment details such as hyperparameters, evaluation procedure, etc., please refer to Section F.

4.1 World Modeling

	Next-Token Test (↑)	Valid Trajectories (↑)	Sequence Compression (↑)	Effective Latent Rank (↓)	Detour Robustness (↑)
GPT	100%	97.0%	0.65	160.1	85.0%
MTP	100%	98.1%	0.64	57.7	**95.0%**
JTP	100%	97.1%	0.32	215.8	87.0%
NextLat	100%	**98.7%**	**0.71**	**52.7**	**95.0%**
True world model	100%	100%	1.00	—	100%

Table 1: Comparison of GPT, MTP, JTP, and NextLat trained on Manhattan taxi rides against the true world model across several metrics.

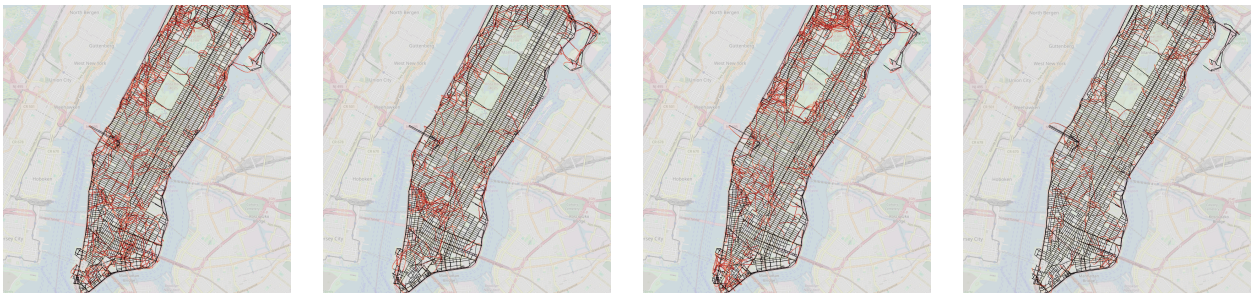
Vafa et al. [2024] introduced a dataset of turn-by-turn taxi rides in Manhattan, where the true world model (i.e., the city's street map) is visually interpretable. Their study revealed that transformers trained on such trajectories can achieve near-perfect next-token accuracy, yet their internal maps remain incoherent; they reconstruct streets with impossible orientations and even flyovers above other roads.

Setup. We use the *random walks* dataset from Vafa et al. [2024], which consists of random Manhattan traversals (91M sequences, 4.7B tokens) between taxi pickup and dropoff points. Models are trained for 6 epochs (vs. 1 epoch in their study) as we observe performance does not converge within a single epoch. Due to its high computational cost, BST is excluded from this benchmark. For JTP, MTP, and NextLat, we set the multi-step prediction horizon at $d = 8$. We evaluate world-modeling performance using five comprehensive metrics:

1. Next-Token Test: Percentage of top-1 token predictions corresponding to legal turns under teacher-forcing on in-distribution validation sequences.
2. Valid Trajectories: Percentage of valid traversals for out-of-distribution (OOD) pickup–dropoff pairs.
3. Sequence Compression: Percentage of cases where the model produces identical continuations when prompted with two different traversals arriving at the same state and sharing the same destination.
4. Effective Latent Rank: Effective rank/dimension of hidden states measured as the exponentiated Shannon entropy of the normalized singular values [Roy and Vetterli, 2007]; lower values indicate better compression.
5. Detour Robustness: Percentage of valid traversals for OOD pickup–dropoff pairs when we substitute the model's top-1 prediction with random detours (legal turns) 75% of the time.

The results are shown in Table 1. More details on training and evaluation are provided in Section F. We also refer motivated readers to Vafa et al. [2024] for further explanations of the next-token test, valid trajectories, sequence compression, and detour robustness metrics.

Results. Similar to the original study, all models achieved 100% accuracy on the next-token test. However, next-token accuracy is a limited diagnostic and cannot meaningfully assess the quality of a model's learned world model. In Figure 3, we visualize each model's internal map using the reconstruction algorithm proposed by Vafa et al. [2024]. Visibly, the transformer trained with NextLat exhibits an internal map more consistent with the true world model.



(a) GPT (b) MTP (c) JTP (d) NextLat (ours)

Figure 3: Reconstructed maps from transformers trained on Manhattan taxi rides using different objectives.

Although not perfect, its inconsistencies (red edges) are sparse and mostly local. Beyond this qualitative evidence, NextLat consistently outperforms all baselines across all metrics. On the trajectory validity and detour robustness metrics, NextLat demonstrates the **strongest generalization to OOD pickup–dropoff pairs**, even when random detours are introduced.

Next, we analyze the compactness of the learned world models using two compression metrics. A model that accurately captures the underlying states and transitions should assign identical continuations to trajectories that end in the same state (i.e., intersection in Manhattan). By this criterion, NextLat achieves the **highest sequence compression of 0.71**. The true Manhattan graph comprises only 4,580 intersections and 9,846 edges, and therefore an effective world model should require only a modest latent dimensionality. Indeed, NextLat has the **lowest effective latent rank of 52.7—over 3x smaller than GPT's**. The combination of stronger planning performance and more compact latent representations reinforces the view that NextLat, by promoting belief state representations and coherent latent dynamics, enables transformers to learn substantially better world models—ones that are both accurate in their predictive structure and efficient in their internal representation of the environment.

4.2 Reasoning

Countdown [Countdown, 2025] is a mathematical reasoning task and a generalized version of the Game of 24, which even frontier models such as GPT-4 [Achiam et al., 2023] have struggled with, achieving 4% by default [Yao et al., 2023]. The goal of the task is to combine a set of given numbers with basic arithmetic operations $(+, -, \times, \div)$ to obtain a target number. For example, given the numbers $\{90, 8, 20, 50\}$, the target number 24 can be obtained using the following sequence of equations: $90 \times 8 = 720$, $50 - 20 = 30$, $720 \div 30 = 24$. Countdown poses a difficult combinatorial search problem due to its large branching factor and the need to efficiently explore the solution space to reach the target number.

Setup. Following Gandhi et al. [2024], we generate 500k training problems with target numbers ranging from 10 to 100 and reserve 10% of the targets for out-of-distribution evaluation. During both training and testing, we insert eight 'pause tokens' [Goyal et al., 2023] after the target number, allowing models additional computation to plan before generating a solution. Performance is measured as the percentage of 10k test problems for which a model produces a valid sequence of equations that correctly reaches the target number. The reported results average over three random seeds per baseline.

Model	Horizon (d)	Accuracy (%)
GPT	–	33.1
BST	–	42.3
MTP	1	39.2
	4	49.7
	8	57.3
JTP	1	39.0
	4	49.4
	8	55.0
NextLat	1	54.8
	4	<u>57.6</u>
	8	**58.7**

Figure 4: Performance on Countdown. Best result is **bolded**, and second best is <u>underlined</u>.